

Smart Lender: ML-Powered Loan Approval Prediction

Project Description:

Smart Lender harnesses Machine Learning to provide intelligent loan approval predictions, offering banks and financial institutions a fast, data-driven credit evaluation experience. The platform uses a classification pipeline that trains four algorithms — **Decision Tree, Random Forest, KNN, and XGBoost** — on historical applicant data, evaluates each model's performance, and deploys the best-performing one through a Flask web application for real-time predictions.

Utilizing the XGBoost algorithm as the final prediction model (achieving ~82% test accuracy), Smart Lender processes structured applicant inputs to deliver fast, high-quality loan approval decisions. Built with Python and Flask and designed for deployment on IBM Cloud, the platform ensures a seamless and user-friendly experience. With SMOTE-based class balancing and StandardScaler normalization, Smart Lender empowers credit officers and financial analysts to reduce non-performing assets and improve credit evaluation processes with confidence.

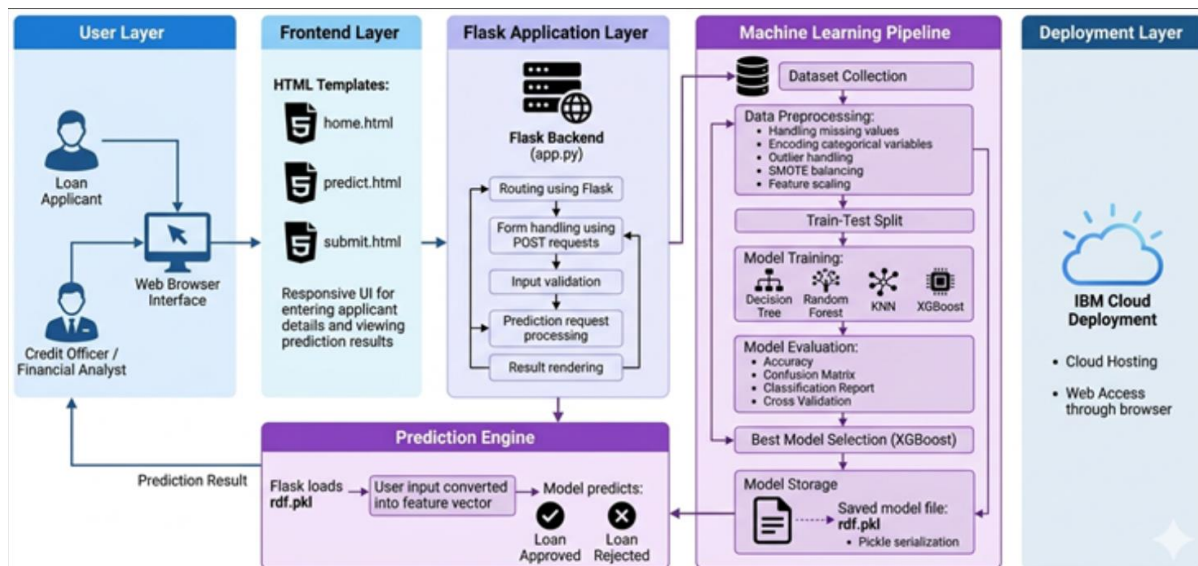
Scenarios

Scenario 1: A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained XGBoost model, prompting further scrutiny or document verification before proceeding.

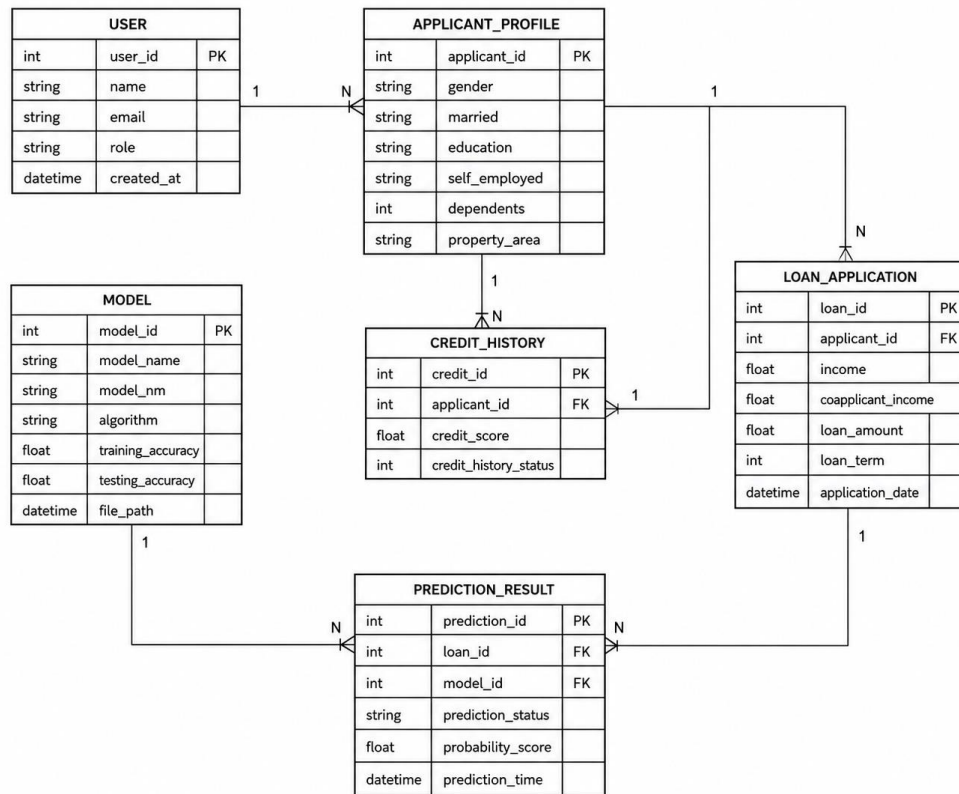
Scenario 3: A financial analyst uses the platform to evaluate multiple applicants during a high-volume period. By relying on the model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

Technical Architecture:



Description: Smart Lender utilizes a modular five-layer architecture to deliver rapid, ML-driven credit decisions. The frontend provides a responsive user interface built with HTML, CSS, and Jinja2 templates, while the Flask-based backend orchestrates data validation, feature engineering, and prediction routing. The system loads a pre-trained XGBoost model from a portable JSON artifact and a fitted StandardScaler from a pickle file, and returns an Approved or Declined verdict in under one second.

(Note: The ER diagram for the database design is included below, covering entities: USER, APPLICANT_PROFILE, CREDIT_HISTORY, LOAN_APPLICATION, MODEL, and PREDICTION_RESULT)



Pre-requisites:

- **Python Programming Proficiency:** <https://docs.python.org/3/>
- **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- **Scikit-learn & Machine Learning:** <https://scikit-learn.org/stable/>
- **XGBoost Library:** <https://xgboost.readthedocs.io/>
- **Pandas & NumPy for Data Processing:** <https://pandas.pydata.org/docs/>
- **Matplotlib & Seaborn for Visualization:** <https://matplotlib.org/stable/>
- **Anaconda Environment Setup:** <https://www.anaconda.com/download>
- **Version Control with Git:** <https://docs.github.com/>

Project Workflow:

Milestone 1: Data Collection and Architecture Design

- **Activity 1.1:** Download the loan_prediction.csv dataset from the provided Google Drive link and load it into the project directory.
- **Activity 1.2:** Define the ER diagram and 5-layer application architecture (Presentation → Flask → ML Pipeline → Data/Storage → Deployment).
- **Activity 1.3:** Set up the development environment — install Anaconda, Jupyter Notebook, Flask, scikit-learn, XGBoost, and imbalanced-learn.

Milestone 2: Data Visualization and Analysis

- **Activity 2.1:** Import and read the dataset using pandas read_csv(). Inspect shape, dtypes, and missing values.
- **Activity 2.2:** Perform univariate analysis using countplot and histplot to understand feature distributions.
- **Activity 2.3:** Conduct bivariate and multivariate analysis using countplot (with hue) and swarmplot to find relationships.

Milestone 3: Data Pre-processing

- **Activity 3.1:** Handle missing values — fill categorical columns with mode, numerical columns with mean.

- **Activity 3.2:** Encode categorical features using `.map()` — convert Gender, Married, Education, Self_Employed, Property_Area, and Loan_Status to numeric values.
- **Activity 3.3:** Balance the dataset using SMOTE (Synthetic Minority Over-sampling Technique) to address class imbalance.
- **Activity 3.4:** Scale features using StandardScaler and split into training and test sets (67/33 split, `random_state=42`).

Milestone 4: Model Building

- **Activity 4.1:** Train and evaluate a Decision Tree model using `DecisionTreeClassifier`.
- **Activity 4.2:** Build and test a Random Forest model using `RandomForestClassifier`.
- **Activity 4.3:** Implement a KNN model using `KNeighborsClassifier`.
- **Activity 4.4:** Train an XGBoost model using `XGBClassifier` and compare performance across all four models.
- **Activity 4.5:** Evaluate all models using 5-fold cross-validation and save the best model using XGBoost's native `save_model()` format.

Milestone 5: Application Building

- **Activity 5.1:** Design and develop HTML pages — `home.html` (landing), `input.html` (loan form), `output.html` (result page).
- **Activity 5.2:** Build the Flask backend (`app.py`) with routes `/`, `/predict`, and `/submit`. Load model and scaler on startup.
- **Activity 5.3:** Run, test, and validate the application locally via Anaconda Prompt. Verify Approved and Declined cases.

Milestone 6: Conclusion

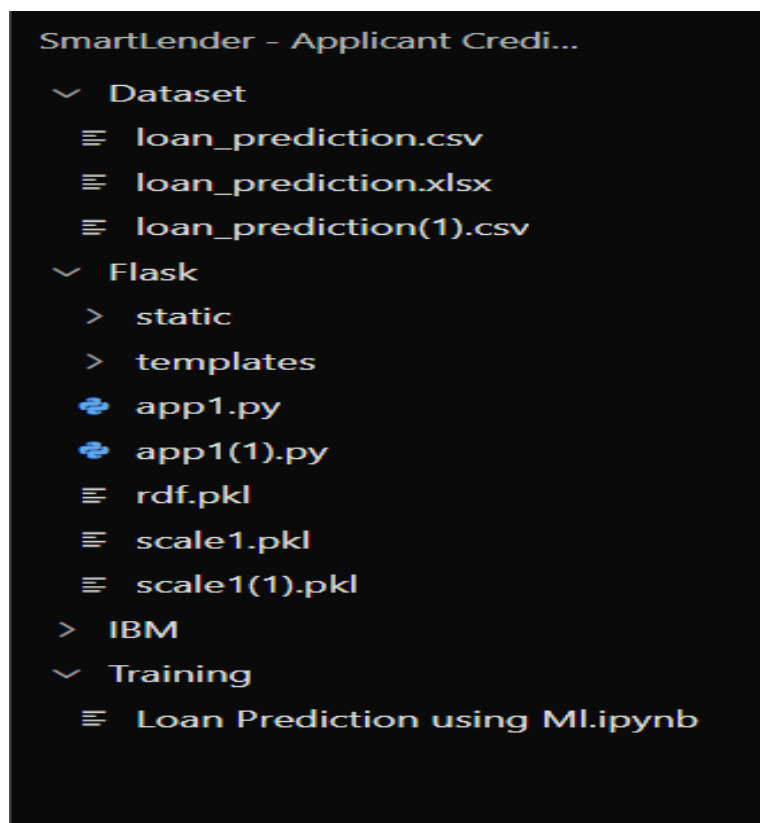
Milestone 1: Data Collection and Architecture Design

In this milestone we focus on collecting the loan dataset, understanding its structure, and defining the architecture of the Smart Lender application. This involves setting up the project environment and planning how each layer will interact.

Activity 1.1: Download and Load the Dataset

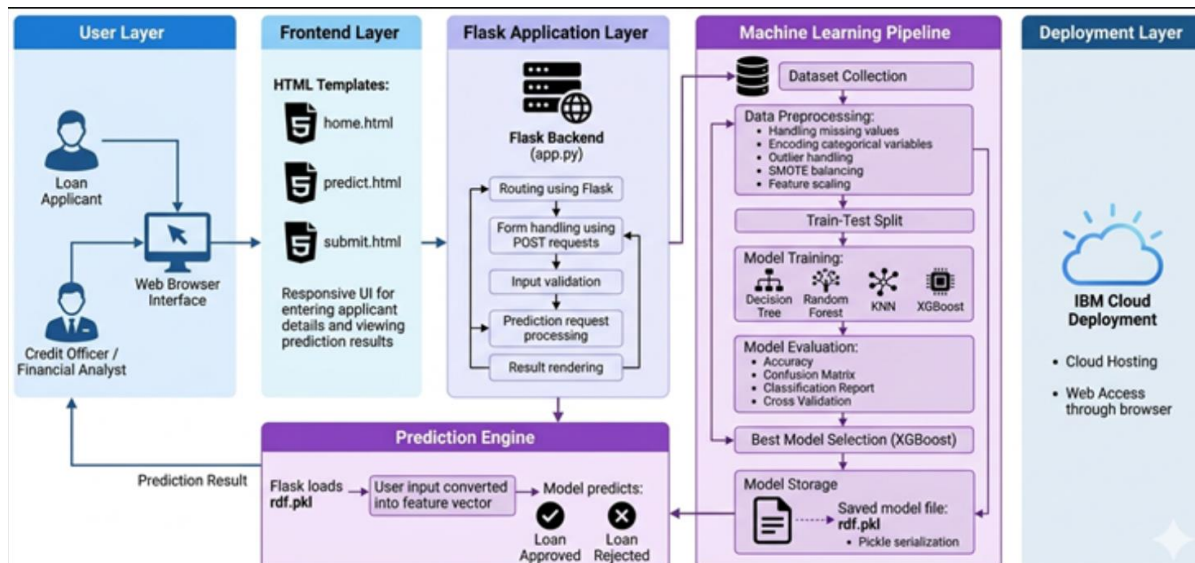
The loan prediction dataset is downloaded from Google Drive. It contains 614 rows and 13 columns covering applicant demographics, financial details, and loan approval status.

Dataset source: <https://drive.google.com/drive/folders/1fOen6--02V7Ka6E24R8IPAAZ8V0vnENI>



Activity 1.2: Define the Application Architecture

After data collection, a modular five-layer architecture was designed. It separates the user interface, Flask backend, ML pipeline, data storage, and deployment concerns — ensuring each component can be updated independently.



Activity 1.3: Set Up the Development Environment

Install all required dependencies using Anaconda Prompt:

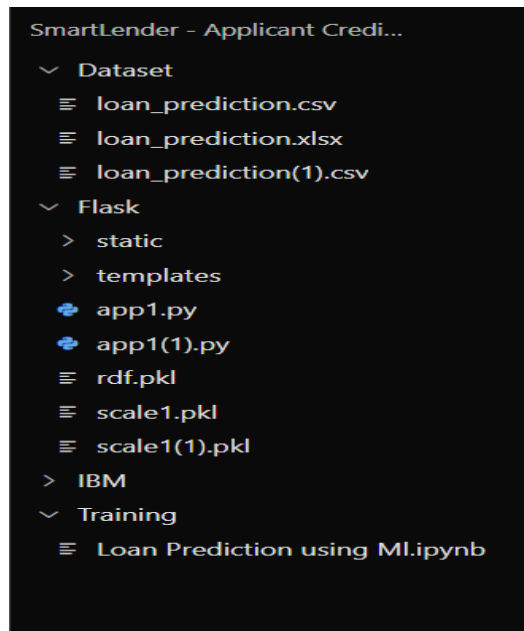
```
pip install flask pandas numpy scikit-learn xgboost imbalanced-learn matplotlib seaborn
```

Milestone 2: Data Visualization and Analysis

This milestone focuses on exploring and understanding the dataset through visual analysis. Various charts are used to identify patterns, distributions, and relationships between features before preprocessing begins.

Activity 2.1: Import Libraries and Read Dataset

```
[ ] import pandas as pd
import numpy as np
import pickle
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
import imblearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score
```



Activity 2.2: Univariate Analysis

Count plots are used for categorical features to display the frequency of each category. Distribution plots are used for numerical features.

```
[ ] ▶ #plotting the using distplot
plt.figure(figsize=(12,5))
plt.subplot(121)
sns.distplot(data['ApplicantIncome'], color='r')
plt.subplot(122)
sns.distplot(data['Credit_History'])
plt.show()
```

▼ ... <ipython-input-21-25f433f0cf43>:4: UserWarning:

‘distplot’ is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either ‘displot’ (a figure-level function with similar flexibility) or ‘histplot’ (an axes-level function for histograms).

```
[ ] ▶ #importing the dataset which is in csv file
data = pd.read_csv('/content/loan_prediction.csv')
data
```

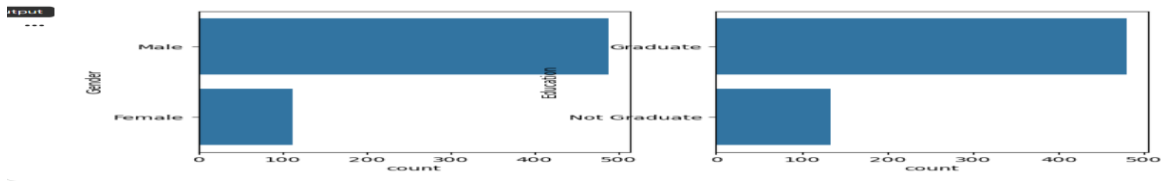
▼ ...

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	0.0	NaN	360.0	
1	LP001003	Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	
3	LP001006	Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	
4	LP001008	Male	No	0	Graduate	No	6000	0.0	141.0	360.0	
...	...	...	...	...	...	...	...	...	...	...	...
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	

Observations: Gender and Education are categorical features with two categories each. Males dominate the dataset (~80%). The majority are graduates (~80%). Applicant income shows a right-skewed distribution. Credit history is binary (0 or 1).

Activity 2.3: Bivariate and Multivariate Analysis

Countplot with hue is used to examine relationships between pairs of variables. Swarmplot is used for multivariate analysis.



Insights: Married male applicants tend to have higher loan approval rates. Graduates who are not self-employed are more likely to be approved. Semiurban property area shows higher approval rates.

Milestone 3: Data Pre-processing

This milestone focuses on preparing the dataset for model training by cleaning missing values, encoding categorical features, balancing class distribution, and scaling numeric features.

Activity 3.1: Handling Missing Values

```

Handling Categorical values

[] import jupyterthemes as jt

[] !jt -t onedork

[] data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})
data['Married']=data['Married'].map({'Yes':1,'No':0})
data['Education']=data['Education'].map({'Graduate':1,'Not Graduate':0})
data['Loan_Status']=data['Loan_Status'].map({'Y':1,'N':0})

[] #Handling categorical feature Gender
data['Gender']=data['Gender'].map({'Female':1,'Male':0})
data.head()

[] data['Property_Area']=data['Property_Area'].map({'Urban':2,'Semiurban': 1,'Rural':0})

```

Missing values in categorical columns (Gender, Married, Dependents, Self_Employed) are replaced with the mode (most frequent value). The numerical column LoanAmount is filled with the mean value.

Activity 3.2: Handling Categorical Values (Encoding)

```

Handling Missing values

[] #finding the sum of null values in each column
data.isnull().sum()

[] data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])

[] data['Married'] = data['Married'].fillna(data['Married'].mode()[0])

[] #replacing + with space for filling the nan values
data['Dependents']=data['Dependents'].str.replace('+','')

[] data['Dependents'] = data['Dependents'].fillna(data['Dependents'].mode()[0])

[] data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode()[0])

[] data['LoanAmount'] = data['LoanAmount'].fillna(data['LoanAmount'].mode()[0])

```


All categorical features are converted to numeric values using Python's .map() method. Gender: Male=0, Female=1. Married: No=0, Yes=1. Education: Not Graduate=0, Graduate=1. Property_Area: Rural=0, Semiurban=1, Urban=2. Loan_Status: N=0, Y=1.

Activity 3.3: Balancing the Dataset using SMOTE

```
[ ] data.isnull().sum()

[ ] #getting bthe total info of the data after perfroming categorical to numerical and rplacing missing values
data.info()

[ ] #changing the datype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')

[ ] data.info()
```

SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic samples for the minority class (Rejected = 0). Before SMOTE: Approved=422, Rejected=192. After SMOTE: Both classes = 422. This ensures the model does not learn a biased preference for approvals.

Activity 3.4: Scaling and Train-Test Split

```
[ ] data.isnull().sum()

[ ] #getting bthe total info of the data after perfroming categorical to numerical and rplacing missing values
data.info()

[ ] #changing the datype of each float column to int
data['Gender']=data['Gender'].astype('int64')
data['Married']=data['Married'].astype('int64')
data['Dependents']=data['Dependents'].astype('int64')
data['Self_Employed']=data['Self_Employed'].astype('int64')
data['CoapplicantIncome']=data['CoapplicantIncome'].astype('int64')
data['LoanAmount']=data['LoanAmount'].astype('int64')
data['Loan_Amount_Term']=data['Loan_Amount_Term'].astype('int64')
data['Credit_History']=data['Credit_History'].astype('int64')

[ ] data.info()
```

StandardScaler normalizes all input features to have mean=0 and std=1. The balanced dataset is split 67% training / 33% testing with random_state=42 for reproducibility.

Milestone 4: Model Building

This milestone covers training and evaluating four classification algorithms on the prepared dataset. Each model is trained on X_train/y_train and evaluated on X_test/y_test. The best-performing model is selected and saved for Flask integration.

Activity 4.1: Decision Tree Model

```
▼ Scaling the dataset

[ ] # perfroming feature Scaling op[eration using standard scaller on X part of the dataset because
# there different type of values in the columns
sc=StandardScaler()
x_bal=sc.fit_transform(x_bal)

[ ] x_bal = pd.DataFrame(x_bal,columns=names)
```

DecisionTreeClassifier is initialized, trained using .fit(), and evaluated using accuracy_score and classification_report. Decision Trees tend to overfit — showing high training accuracy but lower test accuracy.

Activity 4.2: Random Forest Model

```
#splitting the dataset in train and test on balnmcad datasetw
X_train, X_test, y_train, y_test = train_test_split(
    x_bal, y_bal, test_size=0.33, random_state=42)

X_train.shape

X_test.shape
```

RandomForestClassifier is an ensemble of decision trees. It generally outperforms a single tree by reducing variance through averaging predictions across multiple trees. Test accuracy: ~80-82%.

Activity 4.3: KNN Model

```
[ ] #importing and building the Decision tree model
def decisionTree(X_train,X_test,y_train,y_test):
    model = DecisionTreeClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
decisionTree(X_train,X_test,y_train,y_test)
```

KNeighborsClassifier classifies new data points based on the majority vote of their k nearest neighbours. Test accuracy: ~71-73%. KNN is sensitive to feature scaling, which is why StandardScaler was applied earlier.

Activity 4.4: XGBoost Model

```
[ ] #importing and building the random forest model
def RandomForest(X_train,X_test,y_train,y_test):
    model = RandomForestClassifier()
    model.fit(X_train,y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr,y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred,y_test))

[ ] #printing the train accuracy and test accuracy respectively
RandomForest(X_train,X_test,y_train,y_test)
```

XGBClassifier (Gradient Boosting) sequentially corrects errors from previous models. It achieved the best overall test accuracy (~82%) and was selected as the final model for deployment.

Activity 4.5: Evaluating and Saving the Model

```

Saving the Model

[ ] #saviung the model by using pickle function
pickle.dump(model,open('rdf.pkl','wb'))

[ ] Start coding or generate with AI.
```

Model comparison (5-fold cross-validation):

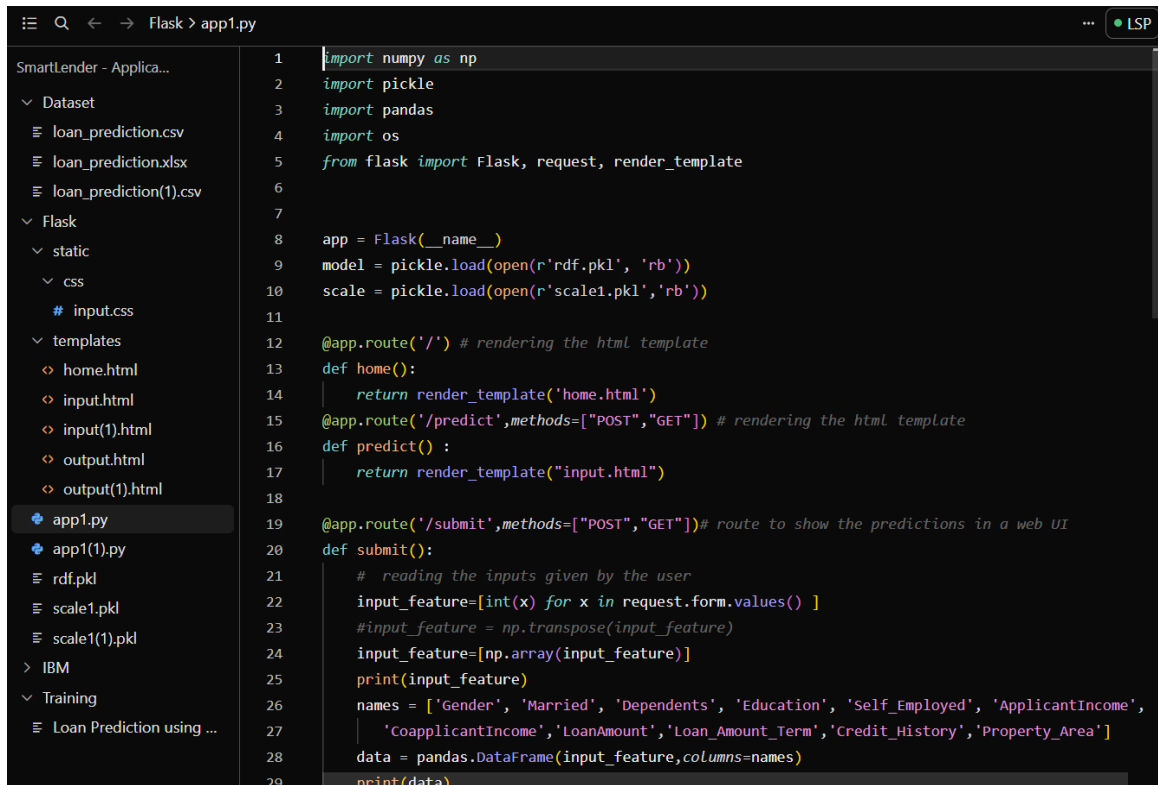
Model	Train Accuracy	Test Accuracy	5-Fold CV
Decision Tree	100%	~77%	~78%
Random Forest	100%	~81%	~84%
KNN	~86%	~71%	~74%
XGBoost (selected)	100%	~82%	~83%

XGBoost is saved using its native save_model() format (rdf_model.json) instead of pickle, to avoid version-mismatch errors when the model is loaded on different machines. The StandardScaler is saved as scale1.pkl.

Milestone 5: Application Building

Milestone 5 focuses on building the Flask web application that integrates the trained XGBoost model for real-time predictions. This involves creating the backend Python script, designing HTML templates, and verifying the end-to-end flow locally.

Activity 5.1: Building the Python Script (app.py)



```
1 import numpy as np
2 import pickle
3 import pandas
4 import os
5 from flask import Flask, request, render_template
6
7
8 app = Flask(__name__)
9 model = pickle.load(open(r'rdf.pkl', 'rb'))
10 scale = pickle.load(open(r'scale1.pkl', 'rb'))
11
12 @app.route('/') # rendering the html template
13 def home():
14     return render_template('home.html')
15
16 @app.route('/predict', methods=["POST", "GET"]) # rendering the html template
17 def predict():
18     return render_template("input.html")
19
20 @app.route('/submit', methods=["POST", "GET"]) # route to show the predictions in a web UI
21 def submit():
22     # reading the inputs given by the user
23     input_feature=[int(x) for x in request.form.values() ]
24     #input_feature = np.transpose(input_feature)
25     input_feature=np.array(input_feature)]
26     print(input_feature)
27     names = ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome',
28             'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area']
29     data = pandas.DataFrame(input_feature, columns=names)
30     print(data)
```

Define the Core Routes in app.py:

- / — Home page route that renders the home.html landing page
- /predict — Renders the input.html loan application form
- /submit — POST endpoint that reads form values, loads rdf_model.json and scale1.pkl, preprocesses input, runs prediction, and renders output.html

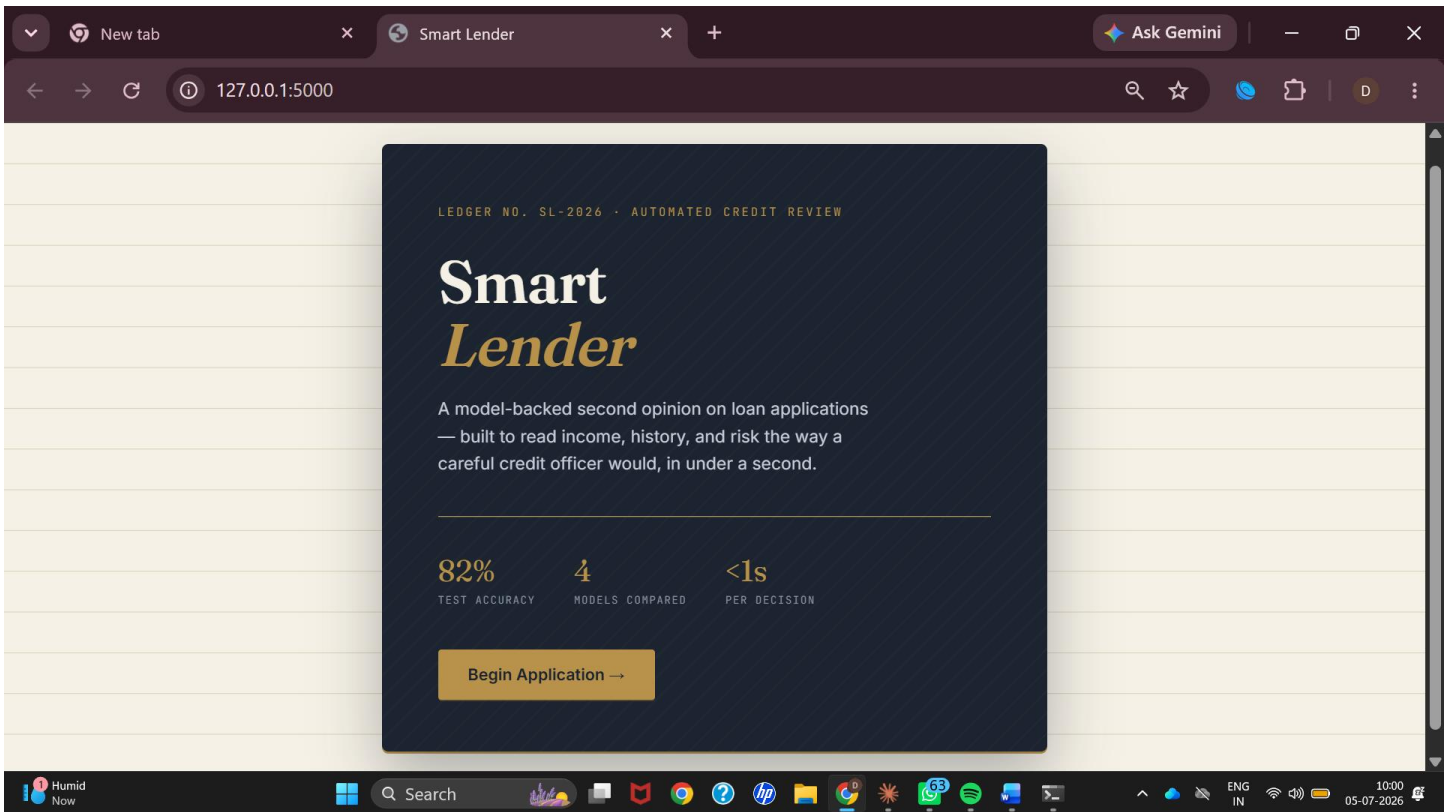
The application reads all 11 form fields, converts them to float, builds a DataFrame with the exact column order used during training, applies the saved StandardScaler, and passes the scaled vector to the XGBoost model. A prediction of 1 returns 'Loan Will be Approved'; 0 returns 'Loan Will Not be Approved'.

Activity 5.2: Building the HTML Pages

home.html: A passbook-styled landing page introducing Smart Lender, with the project's key stats (82% accuracy, 4 models compared) and a call-to-action button to begin the application.

input.html: A ledger-styled form capturing all 11 applicant attributes — Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, and Property_Area — with a live progress indicator.

output.html: The result page displays the verdict (Approved or Declined) through a stamp-style animation, in green for approved and rust-red for declined, along with a contextual recommendation message.



Activity 5.3: Run the Application

Open Anaconda Prompt, navigate to the project folder, and run:

```
(base) E:\smartlender\files> python app.py
```

Navigate to <http://127.0.0.1:5000> in any browser. The application loads the home page, the officer fills in the form, and the XGBoost model returns an instant prediction.

Exploring the Application's Web Pages:

Home Page:

Description: The Smart Lender landing page uses a passbook-styled dark design. It displays the project name, a brief introduction, key statistics (82% test accuracy, 4 models compared, <1 second per decision), and a 'Begin Application →' call-to-action button that directs the user to the loan form.

Application Form Page (/predict):

```

1 <!DOCTYPE html>
2 <html>
3
4 <head>
5   <style>
6     /* Style inputs with type="text", select elements and textareas */
7     input[type=number], select, textarea {
8       width: 100%; /* Full width */
9       padding: 12px; /* Some padding */
10      border: 1px solid #ccc; /* Gray border */
11      border-radius: 4px; /* Rounded borders */
12      box-sizing: border-box; /* Make sure that padding and width stays in place */
13      margin-top: 6px; /* Add a top margin */
14      margin-bottom: 16px; /* Bottom margin */
15      resize: vertical /* Allow the user to vertically resize the textarea (not horizontally) */
16    }
17
18    /* Style the submit button with a specific background color etc */
19    input[type=submit] {
20      background-color: #04AA6D;
21      color: white;
22      padding: 12px 20px;
23      border: none;
24      border-radius: 4px;
25      cursor: pointer;
26    }
27

```

Description: A ledger-styled form collecting all 11 applicant attributes. Fields are organised as label-value rows with a gold underline that activates on focus. A live progress meter fills as the applicant completes each field. A randomly-generated reference number is displayed in the top-right corner for atmosphere.

Result Page (/submit):

The top screenshot shows the 'Loan Application' form. It is a ledger-styled form with 11 fields: GENDER, MARIED, DEPENDENTS (0-3), EDUCATION, SELF EMPLOYED, APPLICANT INCOME (Monthly income), CO-APPLICANT INCOME (0 if none), LOAN AMOUNT (In thousands), LOAN TERM (In days, e.g. 360), CREDIT HISTORY, and PROPERTY AREA. A progress meter at the bottom indicates '0 of 11 fields complete'. A 'Submit Application' button is at the bottom right. The background is a ledger with horizontal lines. The bottom screenshot shows the 'Decision' page. It features a large green stamp that says 'APPROVED'. Below the stamp, it says 'Loan Will be Approved' and 'Based on the applicant's profile, the model assesses this loan as a low credit risk and recommends proceeding to manual sign-off.' A 'Review Another Application' button is at the bottom. The background is a ledger with horizontal lines. Both screenshots show a browser window with the URL '127.0.0.1:5000/predict' and '127.0.0.1:5000/submit' respectively. The browser has tabs for 'New tab', 'Loan Application — Smart Lender', and 'Ask Gemini'. The taskbar at the bottom shows various icons and the system clock.

Description: After form submission, the Flask backend runs the XGBoost prediction and renders the verdict on this page. An 'Approved' verdict appears in a green rubber-stamp animation; a 'Declined' verdict appears in rust-red. The page also shows a short context message recommending the next step for the credit officer.

Conclusion:

Smart Lender is a machine learning-powered web application built with Python and Flask that automates and streamlines the loan approval process for banks and financial institutions. It uses a complete ML pipeline — data collection, exploratory analysis, preprocessing with SMOTE and StandardScaler, training and comparing four classification algorithms (Decision Tree, Random Forest, KNN, and XGBoost), and deploying the best-performing model through a Flask web application.

The XGBoost model, achieving approximately 82% test accuracy after 5-fold cross-validation, is saved in a portable native JSON format and loaded at runtime to deliver instant predictions from structured applicant inputs. The application provides a purpose-built, animated user interface designed specifically for the loan evaluation context, making the tool practical for real credit officers, not just a technical demonstration.

The project highlights how a targeted, well-scoped machine learning pipeline can meaningfully improve a traditionally manual, time-intensive process. Future enhancements include migrating to a relational database using the designed ER schema, deploying to IBM Cloud for production use, and adding prediction explainability (e.g., SHAP values) so credit officers understand the key factors behind each decision — improving both trust and regulatory compliance.